DATA ENGINEERING (CO5173) – FINAL PROJECT
Search Techniques in Modern Database Systems: Elasticsearch 8.15 & Apache Cassandra 5.0

Supervisor: Assoc. Prof. Trần Minh Quang
Students: Đặng Kiến Tường (2670391) – Đặng Minh Quân (2670358)

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology (HCMUT)
Vietnam National University Ho Chi Minh City (VNU-HCM)

Semester 261 – Academic Year 2026–2027

Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch 8.15: Distributed Full-Text & Hybrid Search
3. Apache Cassandra 5.0: Scalable Wide-Column Store & SAI
4. Modern Research Frontier & Combined Architecture
5. Experimental Demo & Quantitative Benchmark ($N = 50,000$)
6. Conclusion



Team Members & Contribution Matrix

Project Module	Đặng Kiến Tường (2670391)	Đặng Minh Quân (2670358)	Weight
1. Search Theory & Math Foundations	Inverted Index, FST, Skip Lists, Roaring Bitmaps, Okapi BM25 derivation & manual verification	Consistent Hashing, LSM-Tree mechanics, HNSW vs. DiskANN/JVector & Reciprocal Rank Fusion (RRF)	20%
2. Elasticsearch 8.15 Engine & Queries	Custom analyzers (`english_custom`, `edge_ngram`), `index_mapping.json`, 10 live ES queries, DFS IDF analysis	HNSW `dense_vector` mapping, hybrid lexical + kNN search tuning, Kibana Dev Tools validation	25%
3. Cassandra 5.0 Engine & SAI	Partition/clustering physical schema, Wide-Partition bucketing & Compaction (`STCS`/`LCS`/`UCS`)	SAI Trie, BKD-Tree & JVector `ANN OF` indexes, `nodetool flush`, SSTable `.db` file inspection	25%
4. Data Pipeline & Benchmark Suite	`generate_data.py` ($N = 50,000$ products, variable-length text, 8D vectors), Docker Compose stack	`benchmark.py` (p_{50}/p_{95} latency suite & $5\text{K} \rightarrow 50\text{K}$ scaling stress test), CDC architecture	30%
Overall Contribution	**50% (Co-Lead: Sections 1, 2 & Demo Part A)**	**50% (Co-Lead: Sections 3, 4, 5 & Demo Part B)**	**100%**

Alignment with CO5173 Course Learning Outcomes

- **L.O.1 (Conceptual, Logical & Physical DB Design):** Contrasts Document-First inverted indexing vs. Query-First wide-column partitioning.
- **L.O.2 (Data Integration Architecture):** Designs a fault-tolerant polyglot architecture syncing Cassandra (Source of Truth) to Elasticsearch via Kafka CDC.
- **L.O.3 (Modern Paradigms & Research):** Evaluates 2026-era hybrid lexical + dense vector search (HNSW vs. Cassandra 5.0 SAI JVector) on $50,000$ records.

Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch 8.15: Distributed Full-Text & Hybrid Search
3. Apache Cassandra 5.0: Scalable Wide-Column Store & SAI
4. Modern Research Frontier & Combined Architecture
5. Experimental Demo & Quantitative Benchmark ($N = 50,000$)
6. Conclusion



Motivation: Why Search is Critical in Modern Systems

Modern Data Scale & User Expectations

- **E-Commerce Catalogs:** Millions of products; users expect sub-20ms responses, typo tolerance, faceted filters, and semantic ranking.
- **Observability & Logs:** Petabytes of semi-structured events requiring real-time ad-hoc text search and aggregations.
- **Social Media & Messaging:** Billions of messages indexed simultaneously by user/channel key and full-text content.

Why Traditional RDBMS Struggle

- `WHERE description LIKE '%laptop%'` forces a **full table scan** $O(N)$.
- B^+-trees only index exact left-prefixes (`'laptop%'`), not arbitrary interior tokens.
- No built-in **relevance scoring** (all matching rows returned with equal weight).
- Vertical scaling bottlenecks under heavy read/write concurrency.

Core Question: How do **Elasticsearch 8.15** (search engine) and **Cassandra 5.0** (wide-column store) solve retrieval at scale?



Search Fundamentals: Inverted Index & Ranking Formulas

Inverted Index Structure Zobel and Moffat, "Inverted Files for Text Search Engines"

- Maps each unique **term** (in a sorted Term Dictionary / FST) to a **posting list** of document IDs, term frequencies, and positions.

Term	DF	Posting List (Doc : TF)
`camera`	8,461	$[D_{12} : 2,\ D_{45} : 1, \dots]$
`grinder`	3,716	$[D_{5212} : 2,\ D_{6010} : 1, \dots]$
`waterproof`	6,140	$[D_{104} : 1,\ D_{22775} : 1, \dots]$

- **FST (Finite State Transducer):** Compresses term dictionary in RAM for fast prefix and fuzzy automaton traversal.
- **Skip Lists & Roaring Bitmaps:** Accelerate boolean conjunctions ($\cap$) and cached filter bitsets.

From TF-IDF to Okapi BM25 Robertson and Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond"

- **TF-IDF:** $\text{TF}(t, d) \times \text{IDF}(t)$ grows unboundedly with term frequency.
- **BM25** adds **term saturation** (k_1) and **document length normalization** (b):

$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\textbf{avgdl}}\right)}$$

$$\text{IDF}(t) = \ln\left(1 + \frac{N - n(t) + 0.5}{n(t) + 0.5}\right)$$

- $k_1 = 1.2$: controls TF saturation asymptote (note: Lucene 8+ folds $(k_1 + 1) = 2.2$ into boost).
- $b = 0.75$: penalizes verbose documents ($|D| >$ avgdl).

Table of Contents

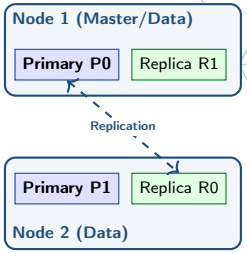
1. Introduction & Search Fundamentals
2. Elasticsearch 8.15: Distributed Full-Text & Hybrid Search
3. Apache Cassandra 5.0: Scalable Wide-Column Store & SAI
4. Modern Research Frontier & Combined Architecture
5. Experimental Demo & Quantitative Benchmark ($N = 50,000$)
6. Conclusion



Elasticsearch Overview & Distributed Architecture

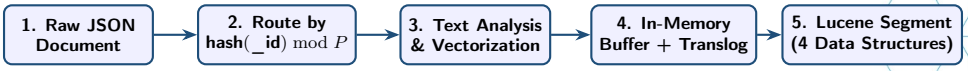
Core Architectural Concepts Gormley and Tong, *Elasticsearch: The Definitive Guide*; Elastic N.V., *Elasticsearch Reference 8.15*

- **Cluster & Nodes:** A collection of nodes coordinating storage and distributed search execution.
- **Index:** Logical namespace mapping to one or more physical shards.
- **Shard (Lucene Instance):** Self-contained Apache Lucene index consisting of immutable **segments**.
  - **Primary Shard:** Handles indexing writes first.
  - **Replica Shard:** Redundant copy for high availability and read throughput scaling.
- **Scatter–Gather Search:** Coordinating node broadcasts query to shards, merges top-K scores.



Each shard combines an Inverted Index + Doc Values + BKD Trees + HNSW Vector Graphs.

Elasticsearch Indexing Pipeline & Multi-Modal Storage



Four Co-Located Per-Segment Structures

1. **Inverted Index (+ FST):** Token $\rightarrow$ sorted posting list of (`doc_id, tf, positions`) for full-text search.
2. **Doc Values (Columnar):** Column-stride arrays on disk (`.dvd/.dvm`) for fast sorting & aggregations.
3. **BKD Trees:** Block K-D trees for $O(\log N)$ numeric, date, and geo-spatial range filtering.
4. **HNSW Graph (`.vec/.vex`):** Multi-layer proximity graph for dense vector k-NN similarity search.

Near Real-Time (NRT) Refresh & Durability

- **In-Memory Buffer + Translog:** New documents are appended to the write-ahead `translog` for crash recovery and buffered in RAM.
- **Refresh (every 1s):** Buffer is written to a new immutable **Lucene segment** in OS page cache $\rightarrow$ document becomes searchable within 1 second.
- **Flush & Tiered Merge:** Periodically commits segments to disk (`fsync`), clears translog, and merges smaller segments in the background while purging deleted docs.

Elasticsearch Analyzers & HNSW Mapping Deep Dive

Index Mapping (demo/es/index_mapping.json):

```
"analysis": {
  "analyzer": {
    "english_custom": {
      "tokenizer": "standard",
      "filter": ["lowercase", "english_stemmer",
           "english_stop"]
    },
    "autocomplete": {
      "tokenizer": "standard",
      "filter": ["lowercase", "edge_ngram_2_15"]
    }
  }
},
"mappings": { "properties": {
  "name": { "type": "text", "analyzer": "english_custom",
    "fields": { "autocomplete": { "type": "text",
      "analyzer": "autocomplete", "search_analyzer":
           "standard" }}},
  "description": { "type": "text", "analyzer":
       "english_custom" },
  "embedding": { "type": "dense_vector", "dims": 8,
    "index": true, "similarity": "cosine",
    "index_options": {"type": "hnsw", "m": 16,
         "ef_construction": 100}}
}}
```

Demo 9: `_analyze` API Transformation

- **Input:** `"The professional cameras are featuring waterproof housings"`
- **1. Standard Tokenizer:** Splits on word boundaries.
- **2. Lowercase + Stop Filter:** Removes `"The"`.
- **3. English Stemmer:** Reduces words to root stems:

```
["profession", "camera", "ar",
 "featur", "waterproof", "hous"]
```

Autocomplete (`edge_ngram_2_15`):

- `"Samsung"` $\rightarrow$ `["sa", "sam", "sams", "samsu", "samsun", "samsung"]` at write time.
- `search_analyzer: "standard"` keeps prefix queries $O(1)$ at read time (1.48 ms over $50,000$ products!).



Elasticsearch Search Types – Part 1: Text, Fuzzy & Autocomplete

1. Full-Text `match` Query (Demo 1)

```
{ "size": 3,
  "query": {
    "match": {
      "description": "lightweight laptop for travel"
    }
  },
  "_source": ["name", "category", "price",
       "rating"] }
```

- Analyzes query into stemmed tokens; ranks $> 10,000$ matching docs in 3.2 ms via BM25 (`max_score`: 7.12).

3. Edge N-Gram Autocomplete (Demo 3)

```
{ "size": 5,
  "query": {
    "match": { "name.autocomplete": "sams" }
  },
  "_source": ["name", "brand", "category"] }
```

2. Fuzzy Multi-Match Query (Demo 2)

```
{ "size": 3,
  "query": {
    "multi_match": {
      "query": "lapptop wireles",
      "fields": ["name^2", "description"],
      "fuzziness": "AUTO"
    }
  },
  "_source": ["name", "category", "price"] }
```

How Fuzzy & Field Boosting Work

- `fuzziness: "AUTO"` constructs deterministic **Levenshtein Automata** (edit distance $d \leq 1$ for 3–5 chars, $d \leq 2$ for > 5 chars) intersected directly with the in-memory FST!
- `"lapptop"` $\rightarrow$ `"laptop"` ($d = 1$); `"wireles"` $\rightarrow$ `"wireless"` ($d = 1$), matching 8,280 docs in 5.6 ms.
- `name^2` doubles relevance weight of title matches.

Elasticsearch Search Types – Part 2: Bool, Aggregations & Function Score

4. Compound `bool` Query (Demo 4)

```
"query": { "bool": {
  "must": [
    { "match": { "description": "noise cancelling"
        } }
  ],
  "filter": [
    { "term":  { "category": "headphones" } },
    { "range": { "price":  { "lte": 100 } } },
    { "range": { "rating": { "gte": 4.0 } } }
  ]
}}
```

- **Query Context (`must`):** Evaluates BM25 relevance score.
- **Filter Context (`filter`):** Binary yes/no (0 scoring overhead); frequent filters are cached in memory as **Roaring Bitmaps** per segment.

5. Faceted Aggregations (Demo 5)

```
"size": 0,
"aggs": { "by_category": {
  "terms": { "field": "category" },
  "aggs": {
    "avg_price":  { "avg": { "field": "price" } },
    "max_rating": { "max": { "field": "rating" } }
  }
}}
```

6. Custom Ranking (`function_score`, Demo 6)

```
"query": { "function_score": {
  "query": { "match": { "description": "camera" } },
  "functions": [
    { "field_value_factor": { "field": "rating",
        "factor": 1.2, "modifier": "sqrt" } },
    { "field_value_factor": { "field":
        "review_count", "factor": 0.001,
        "modifier": "log1p" } }
  ],
  "boost_mode": "multiply"
}}
```

Elasticsearch BM25 Verification: Multi-Term & TF Saturation ($N = 50,000$)

Demo 8: explain: true for "conical burr grinder"


```
"_id": "5212", "_score": 11.602137,
"_source": { "name": "Philips EspressoMaster 222",
  "description": "The Philips EspressoMaster 222 is a
   professional coffee machine (automatic) featuring
   built-in grinder and conical burr grinder.
   Ideal for professionals." },
"_explanation": { "value": 11.602137, "details": [
  { "value": 3.7997048,
    "description": "weight(description:conic) [f=1,
         n=2022]" },
  { "value": 3.7997048,
    "description": "weight(description:burr)  [f=1,
         n=2022]" },
  { "value": 4.0027270,
    "description": "weight(description:grinder) [f=2,
         n=3716]",
    "details": [
      { "value": 2.2, "description": "boost (k1+1)" },
      { "value": 2.5992606, "description": "idf
           (n=3716, N=50000)" },
      { "value": 0.6999765, "description": "tf (f=2,
           dl=15, avgdl=24.23)" }
    ]} ]}
```


Exact Mathematical Verification ($N = 50,000$): Doc _id $= 5212$ has $|D| = 15$ vs. avgdl $= 24.2266$, so length norm $K = 1.2\left(0.25 + 0.75\frac{15.0}{24.2266}\right) = \mathbf{0.85723}$.

- **Terms** conic **&** burr **($f = 1,\ n = 2022$):**

$$\text{IDF} = \ln\left(1 + \frac{50000 - 2022 + 0.5}{2022 + 0.5}\right) = 3.20771$$

$$\text{TF}_{\text{norm}} = \frac{1.0}{1.0 + 0.85723} = 0.53843, \quad S = 2.2 \times \text{IDF} \times \text{TF}_{\text{norm}}$$

- **Term** grinder **($f = 2,\ n = 3716$):**

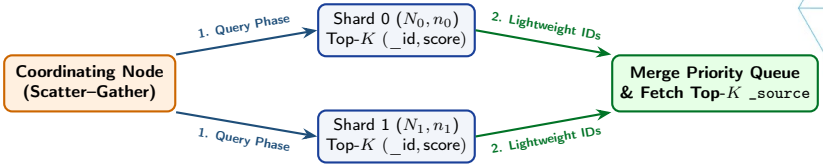
$$\text{IDF} = \ln\left(1 + \frac{50000 - 3716 + 0.5}{3716 + 0.5}\right) = 2.59926$$

$$\text{TF}_{\text{norm}} = \frac{2.0}{2.0 + 0.85723} = 0.69998, \quad S = 2.2 \times \text{IDF} \times \text{TF}_{\text{norm}}$$

Total Score & TF Saturation Insight

Total BM25 $= 3.79970 + 3.79970 + 4.00273 = \mathbf{11.602137}$.
Doubling grinder ($f : 1 \to 2$) raises TF_{norm} by only $+30\%$ ($0.538 \to 0.700$)!

Distributed Search Internals: Two-Phase Execution & Shard IDF Skew



Standard Two-Phase `query_then_fetch`

1. **Query Phase:** Coordinating node fans out query to all P shards. Each shard executes locally and returns *only* top-K lightweight tuples (`_id, _score`).
2. **Fetch Phase:** Coordinator merges $P \times K$ scores, picks global top-K, and requests full `_source` JSON only from the winning shards.

The Local IDF Skew Problem & `dfs_query_then_fetch`

- By default, each shard computes IDF(t) using **local shard statistics** (N_i, n_i) for speed.
- On small or custom-routed indices, term frequencies skew across shards → identical documents on different shards get different BM25 scores!
- **Solution:** `search_type=dfs_query_then_fetch` adds a pre-query round-trip to aggregate global $\sum N_i$ and $\sum n_i$ before scoring.

Table of Contents

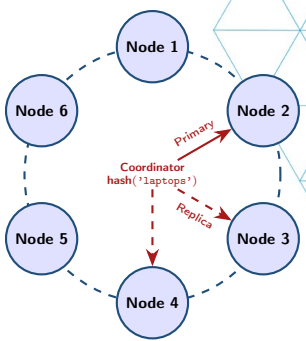
1. Introduction & Search Fundamentals
2. Elasticsearch 8.15: Distributed Full-Text & Hybrid Search
3. **Apache Cassandra 5.0: Scalable Wide-Column Store & SAI**
4. Modern Research Frontier & Combined Architecture
5. Experimental Demo & Quantitative Benchmark ($N = 50,000$)
6. Conclusion



Apache Cassandra Overview: Masterless Ring Architecture

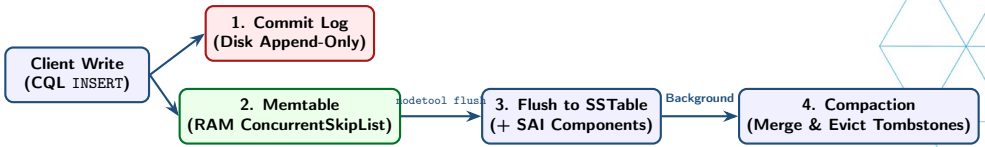
Decentralized Peer-to-Peer Design Lakshman and Malik, "Cassandra: A Decentralized Structured Storage System"; Karger et al., "Consistent Hashing and Random Trees"

- **Masterless Ring Topology:** Every node plays an identical role; no single point of failure (SPOF).
- **Consistent Hashing:** Partition key is hashed via **Murmur3Partitioner** (range $[-2^{63}, 2^{63}-1]$) to determine token ownership on the ring.
- **Virtual Nodes (vnodes):** Each physical node owns multiple token ranges (e.g., 16–256 vnodes) for balanced data distribution and seamless cluster resizing.
- **Coordinator Node:** Any node can receive a client request and route it directly in $O(1)$ hops to the replica nodes holding that token.
- **Tunable Consistency:** Configure `ONE`, `LOCAL_QUORUM`, or `ALL` per query ($R + W >$ RF guarantees strong consistency).



Replication Factor (RF=3): token is replicated clockwise around the consistent hash ring.

Cassandra Write Path & LSM-Tree Storage Engine



Why Writes Are Ultra-Fast ($O(1)$ Disk I/O)

- **No Read-Before-Write:** Unlike B-Trees, Cassandra never seeks or overwrites existing pages in place O'Neil et al., "The Log-Structured Merge-Tree (LSM-Tree)".
- **Dual Write:** Sequential append to the **Commit Log** (durability) + insertion into the in-memory **Memtable** sorted by clustering key (`price ASC, product_id ASC`).

SSTables, Tombstones & Compaction

- **SSTable (Sorted String Table):** Flushed sequentially to disk; completely immutable (`Data.db`, `Index.db`, `Filter.db`).
- **Deletes = Tombstones:** Marker written with timestamp; reclaimed after `gc_grace_seconds`.
- **Compaction:** Periodically merges multiple SSTables into one, keeping only the latest cell timestamp.

Cassandra Read Path: Multi-Layer Filtering & Real SSTable Trace



How Cassandra Avoids Reading Every SSTable

1. **Bloom Filter (Off-Heap RAM):** Bit array per SSTable (`Filter.db`). Guarantees *zero false negatives* — skips SSTables missing the partition key.
2. **Partition Summary (`Summary.db`) & `Index.db`:** Binary-searches sampled keys in RAM to jump directly to the disk offset.
3. **Clustering Slice Optimization (`Statistics.db`):** Min/max clustering metadata skips SSTables outside `price BETWEEN 1000 AND 1500`.

Demo 8: Live `TRACING ON` After `nodetool flush`

Query: `WHERE category='cameras' AND price>=1000 AND price<=1500 LIMIT 3` ($50,000$ rows across 2 flushed SSTables):

- `Executing single-partition query` ($1,811\ \mu$s)
- `Skipped 0/2 non-slice-intersecting sstables` ($2,028\ \mu$s)
- `Partition index found for sstable 4 & 5` ($2,674\ \mu$s)
- `Merged data from memtables and 2 sstables` ($3,345\ \mu$s)
- `Read 3 live rows and 0 tombstone cells` (**3.45 ms total!**)

Cassandra Query-First Data Modeling

Schema Definition (`demo/cassandra/schema.cql`):

```
CREATE KEYSPACE IF NOT EXISTS shop
  WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
  };

CREATE TABLE IF NOT EXISTS shop.products_by_category (
  category      text,
  price         decimal,
  product_id    int,
  name          text,
  brand         text,
  description   text,
  rating        float,
  stock         int,
  review_count  int,
  embedding     vector<float, 8>,
  PRIMARY KEY ((category), price, product_id)
) WITH CLUSTERING ORDER BY (price ASC, product_id ASC);
```

Anatomy of the Compound Primary Key

`PRIMARY KEY ((category), price, product_id)`

- **Partition Key** (`category`): Determines *which node/partition* stores the data via Murmur3(`category`). All $\sim 8,280$ laptops live together in one physical partition.
- **Clustering Columns** (`price, product_id`): Determines the *on-disk sort order* within that partition.
- `product_id` acts as a unique tie-breaker so two products with identical price do not overwrite each other.

Empirical Result ($N = 50,000$): Category browse + price range filter executes in **1.70–2.31 ms (p_{50})** via a single sequential disk read!

Cassandra 5.0 Storage-Attached Indexes (SAI) & On-Disk Layout

3 SAI Modalities in `schema.cql` Apache Software Foundation, *Storage-Attached Indexing (SAI) in Cassandra 5.0*:

```
-- 1. Text Equality (Byte-Ordered Trie)
CREATE CUSTOM INDEX idx_brand
  ON shop.products_by_category (brand)
  USING 'StorageAttachedIndex';

-- 2. Numeric Range (Balanced BKD-Tree)
CREATE CUSTOM INDEX idx_rating
  ON shop.products_by_category (rating)
  USING 'StorageAttachedIndex';

-- 3. Dense Vector ANN (JVector DiskANN Graph)
CREATE CUSTOM INDEX idx_embedding
  ON shop.products_by_category (embedding)
  USING 'StorageAttachedIndex'
  WITH OPTIONS = {'similarity_function': 'COSINE'};
```

Demo 10: Real On-Disk SSTable & SAI Files

```
nb-4-big-Data.db                                (3.6 MB)
nb-4-big-Filter.db                              (24 B
     Bloom)
nb-4-big-Index.db / Summary.db                  (5.5 KB)
nb-4-big-SAI+aa+PartitionKeyBlocks.db           (Shared)
nb-4-big-SAI+aa+ClusteringKeyBlocks.db          (276 KB)
nb-4-big-SAI+aa+idx_brand+TermsData.db          (Trie)
nb-4-big-SAI+aa+idx_brand+PostingLists.db (33 KB)
nb-4-big-SAI+aa+idx_rating+BalancedTree.db(86 KB
     BKD)
nb-4-big-SAI+aa+idx_embedding+CompressedVectors.db
nb-4-big-SAI+aa+idx_embedding+TermsData.db(4.8 MB)
```

Why SAI Beats Legacy 2i

- **Shared Row/Key Offsets:** `PartitionKeyBlocks.db` and `ClusteringKeyBlocks.db` are shared across all SAI indexes on the table (60–80% less disk footprint).
- Flushed alongside SSTables with zero extra write-amplification path.

Advanced Cassandra Tuning: Wide Partitions, Bucketing & Compaction

1. The Wide-Partition Hotspot Problem

- In our schema, Partition Key = (category) puts $\sim 8,300$ rows (~ 2.8 MB) per partition — ideal (< 100 MB and $< 100,000$ cells per partition).
- If an e-commerce catalog grows to **50M products** in 'laptops', a single partition exceeds 15 GB → GC pauses, slow compaction, and single-replica hotspot!

Solution: Synthetic Partition Bucketing

```
-- Split partition by price_bucket (floor(price/500))
-- or hash prefix (product_id % 16):
PRIMARY KEY ((category, price_bucket), price,
     product_id)
```

Distributes 'laptops' across multiple vnodes while preserving sorted price scans inside each bucket!

2. Compaction Strategies & Tombstone Eviction

Strategy	Mechanism	Best Workload
STCS (Default)	Merges similarly sized SSTables ($4 \rightarrow 1$)	Write-heavy, append-only logs
LCS (Leveled)	Fixed-size non-overlapping levels ($L_0, L_1, \ldots$)	Read-heavy point lookups (90% reads hit 1 SSTable)
UCS (Cass 5.0)	Unified density-tiered sharding; auto-adapts W/R	Modern mixed read/write + SAI workloads

- **Tombstone Hazard:** `DELETE` writes a tombstone marker. Until `gc_grace_seconds` (10 days) expires and compaction runs, reads must scan across tombstones (aborting if $> 100,000$ via `tombstone_failure_threshold`).

Cassandra Limitations for Full-Text Search

1. Refusal on Unindexed Columns (Demo 5)

```
SELECT name FROM shop.products_by_category
WHERE stock > 100 LIMIT 5;
-- InvalidRequest: Cannot execute this query as it
-- might involve data filtering... use ALLOW FILTERING
```

2. The `ALLOW FILTERING` Anti-Pattern (Demo 6)

```
SELECT name, category, stock, review_count
FROM shop.products_by_category
WHERE stock = 500 AND review_count > 650
LIMIT 5 ALLOW FILTERING;
```

- Forces coordinator to scan **all 50,000 rows across all SSTables** ($p_{50} = 115.88$ ms vs. 1.70 ms partition key — **68× slower!**).

3. No `LIKE '%...%'` Substring Search (Demo 7)

```
SELECT name FROM shop.products_by_category
WHERE description LIKE '%laptop%' LIMIT 3;
-- InvalidRequest: LIKE restriction is only supported
-- on properly indexed columns.
```

What Cassandra Cannot Do Natively

- **No Lexical Relevance Ranking (BM25):** Rows are returned strictly in token or clustering order — no TF-IDF or document-length normalization.
- **No Fuzzy / Typo Tolerance:** Cannot match `"lapptop"` to `"laptop"` via Levenshtein automata.
- **No Stemming or Stopwords:** `"cameras"` will not match `"camera"` without an analyzer pipeline.
- **No Faceted Aggregations:** Computing `GROUP BY category` with `AVG(price)` across partitions triggers a full-cluster warning.

Key Insight: Cassandra's refusal is a *deliberate guardrail*, protecting cluster SLA from unbounded tail latency.



Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch 8.15: Distributed Full-Text & Hybrid Search
3. Apache Cassandra 5.0: Scalable Wide-Column Store & SAI
4. **Modern Research Frontier & Combined Architecture**
5. Experimental Demo & Quantitative Benchmark ($N = 50,000$)
6. Conclusion



2026 Research Frontier: Dense Vector & Hybrid Search

Approximate Nearest Neighbor (ANN) Graphs

- Lexical BM25 fails on **vocabulary mismatch** (e.g., querying `"portable commuter gear"` vs. product text `"lightweight ultrabook"`).
- Dense embeddings map items to $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ scored by **Cosine Similarity**:

$$\text{sim}_{\cos}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \, \|\mathbf{v}\|_2}, \qquad S_{\textbf{ES}}(\mathbf{u}, \mathbf{v}) = \frac{1 + \text{sim}_{\cos}(\mathbf{u}, \mathbf{v})}{2}$$

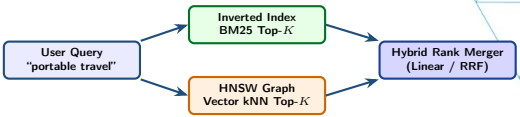
- **Elasticsearch 8.15 (Lucene HNSW, Malkov & Yashunin 2020):** Multi-layer hierarchical skip-graph ($M = 16$, efConstruction $= 100$). Fast $O(\log N)$ greedy routing from coarse top layer to dense bottom layer L_0.
- **Cassandra 5.0 SAI (JVector / DiskANN, Subramanya et al. 2019):** Single-layer **Vamana graph** with Product Quantization (`CompressedVectors.db`), enabling disk-resident billion-scale ANN search without holding full vectors in JVM heap!

Hybrid Search Fusion: Linear Boost vs. RRF

- Combining lexical BM25 ($S_{\textbf{BM25}}$) with semantic vector similarity ($S_{\textbf{kNN}}$) captures both exact SKU/brand matches and **conceptual intent** (Cormack et al., SIGIR 2009):

$$S_{\textbf{Hybrid}}(d) = w_{\textbf{lex}} \cdot S_{\textbf{BM25}}(d) + w_{\textbf{vec}} \cdot S_{\textbf{kNN}}(d)$$

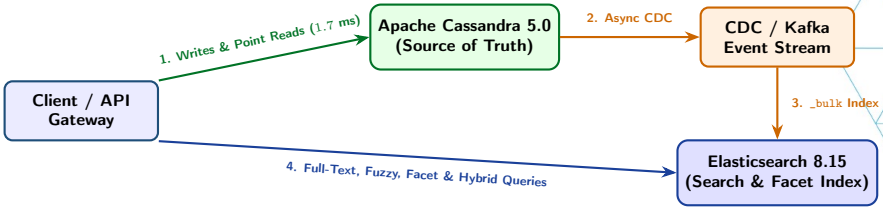
$$\text{RRF}(d) = \sum_{m \in \{\textbf{BM25}, \textbf{kNN}\}} \frac{1}{k + \text{rank}_m(d)} \quad (k = 60)$$



Elasticsearch 8.15 vs. Apache Cassandra 5.0: Side-by-Side Comparison

Dimension	**Elasticsearch 8.15** Elastic N.V., *Elasticsearch Reference 8.15*	**Apache Cassandra 5.0** Apache Software Foundation, *Apache Cassandra 5.0 Documentation*
Primary Paradigm	Distributed Search & Analytics Engine	Wide-Column Distributed NoSQL Store
Core Data Structures	Lucene Inverted Index + FST + Doc Values + BKD Trees + HNSW Vector Graph	Partitioned LSM-Tree (Memtable + Immutable SSTables) + SAI (Trie, BKD, JVector)
Query Capabilities	Full-text BM25, fuzzy Levenshtein automata, edge n-grams, faceted aggregations, Hybrid kNN	$O(1)$ partition key lookup, clustering range scans, exact/range/ANN SAI filtering
Data Modeling	Schema-flexible JSON; index all searchable fields (*Document-First*)	Denormalized tables per access pattern (*Query-First*)
Write Performance	Medium–High (CPU tokenization/HNSW build, 1s NRT refresh delay)	Ultra-High (append-only commit log + RAM memtable, immediate visibility)
Measured Read p_{50}	5.96–12.93 ms HTTP (1.0–7.5 ms engine) across 50K docs	1.70 ms Partition Key, 3.84 ms SAI, 8.31 ms ANN (115.88 ms full scan)
Consistency Model	Eventual read visibility (1s NRT refresh); primary → replica sync writes	Tunable per operation (`ONE`, `LOCAL_QUORUM`, `ALL`) + LWT (`Paxos`/`Accord`)
Ideal Use Case	Product discovery, log analytics, ad-hoc user queries, faceted navigation	High-velocity writes, source-of-truth order/catalog storage, time-series

Combined Production Architecture: Best of Both Worlds



Synchronization Patterns

- **Change Data Capture (CDC) + Kafka:** Cassandra commits writes first; CDC connector streams mutations to Kafka, which bulk-indexes into Elasticsearch using `product_id` as idempotent `_id`.
- **Decoupled Failure Domains:** If Elasticsearch is reindexing or under heavy analytical load, core checkout writes in Cassandra remain 100% unaffected.

Industry Adoption (Netflix, Discord, eBay)

- **E-Commerce Discovery-to-Checkout Flow:**
  1. User types `"noise cancelling headphones"` $\rightarrow$ ES returns ranked `product_ids` & category facets in < 10 ms.
  2. Product detail / Checkout service queries **Cassandra** by key for authoritative real-time stock and price.

Dual-Store Consistency Engineering: Solving the NRT & CDC Lag

The Eventual Consistency Race Condition

- Total sync delay
  $\Delta t = t_{\text{CDC}} + t_{\text{Kafka}} + t_{\text{ES_refresh}} \approx 1.2\text{–}2.5$ seconds.
- **Scenario:** Last unit of `Sony AlphaShot` (stock $= 1$) is purchased at $t = 0$. At $t = 0.5$s, Elasticsearch still shows stock $= 1$! How do we prevent overselling?

Solution: Read-Repair & Authoritative Verification

1. **Search Stage (Eventual Consistency OK):** ES returns top-K candidate IDs (`[26015, 13815, 16040]`).
2. **Hydration Stage (Fast Point Lookup):** API Gateway fetches live price/stock from Cassandra at `LOCAL_QUORUM` (1.7 ms).
3. **Checkout Stage (Linearizable LWT):** Atomic compare-and-set in Cassandra: `UPDATE inventory SET stock = stock - 1 WHERE product_id = 26015 IF stock > 0;`

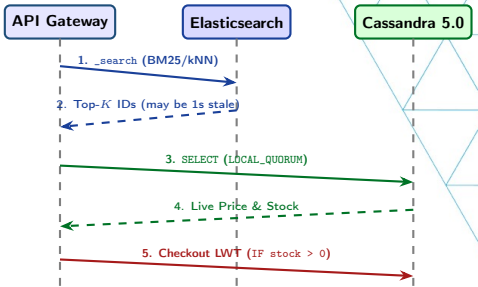


Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch 8.15: Distributed Full-Text & Hybrid Search
3. Apache Cassandra 5.0: Scalable Wide-Column Store & SAI
4. Modern Research Frontier & Combined Architecture
5. **Experimental Demo & Quantitative Benchmark ($N = 50,000$)**
6. Conclusion



Experimental Setup: 50,000-Product Catalog & 8D Embeddings

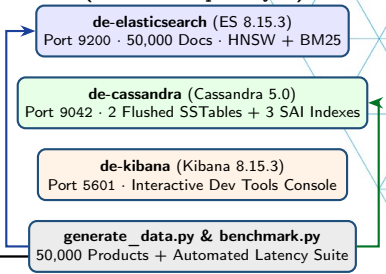
Upgraded Synthetic Dataset (`generate_data.py -count 50000`)

- **Volume:** $N = 50,000$ products loaded identically into Elasticsearch (5 bulk chunks $\times 10$K) and Cassandra (`COPY FROM` + `nodetool flush`).
- **Variable-Length Rich Text:** $|D| \in [12, 44]$ tokens (avgdl $= 24.23$) with natural term repetition ($f \in \{1, 2, 3\}$) for realistic BM25 ranking.
- **8D Unit-Normalized Semantic Vectors (`embedding`):** Encodes `[portability, compute, audio, imaging, battery, waterproof, brew, athletic]`.

Category	Docs	Avg Price ($)	Sample Brands (8 per Category)
`cameras`	8,461	2,181.53	Canon, Nikon, Sony, Fujifilm, GoPro, Leica
`running shoes`	8,366	157.69	Nike, Adidas, Asics, New Balance, Hoka, Salomon
`headphones`	8,328	287.31	Sony, Bose, JBL, Anker, Sennheiser, Beyerdynamic
`smartphones`	8,320	1,001.72	Samsung, Apple, Xiaomi, Sony, Google, OnePlus
`laptops`	8,280	1,985.25	Dell, Asus, Lenovo, Apple, HP, MSI, Acer, Razer
`coffee machines`	8,245	613.51	DeLonghi, Breville, Philips, Nespresso, Jura

Reproducible Docker Stack

(`docker-compose.yml`)



Live Demo: Elasticsearch Full-Text (BM25) & Fuzzy Search ($N = 50,000$)

Demo 1: Full-Text Search (Distinct BM25 Scores)

```
POST /products/_search
{ "size": 3, "query": { "match": {
    "description": "lightweight laptop for travel"
        }},
  "_source": ["name", "category", "price", "rating"]
      }
```

Real Execution Output ($> 10,000$ hits, 3.2 ms engine):

```
"hits": [
  { "_id": "2302",  "_score": 7.119585,
    "_source": { "name": "Razer WorkStation 685",
      "category": "laptops", "price": 3013.5,
           "rating": 4.4 } },
  { "_id": "39545", "_score": 6.905856,
    "_source": { "name": "Apple AirLite 508",
      "category": "laptops", "price": 3494.91,
           "rating": 3.2 } },
  { "_id": "25556", "_score": 6.8578253,
    "_source": { "name": "Apple GameMaster 422",
      "category": "laptops", "price": 1864.08,
           "rating": 3.7 } }
]
```

Demo 2: Fuzzy Search with Typos

```
POST /products/_search
{ "size": 3, "query": { "multi_match": {
    "query": "lapptop wireles",
    "fields": ["name^2", "description"],
        "fuzziness": "AUTO"}},
  "_source": ["name", "category", "price"] }
```

Real Execution Output (8,280 hits, 7.5 ms engine):

```
"hits": [
  { "_id": "10584", "_score": 2.065803,
    "_source": { "name": "Acer AirLite 168",
      "category": "laptops", "price": 1174.06 } },
  { "_id": "11362", "_score": 2.065803,
    "_source": { "name": "Acer BladePro 214",
      "category": "laptops", "price": 1918.26 } },
  { "_id": "45033", "_score": 2.065803,
    "_source": { "name": "MSI BladePro 129",
      "category": "laptops", "price": 1425.37 } }
]
```



Live Demo: Elasticsearch Bool Filters & Faceted Aggregations (50K Docs)

Demo 4: Bool Query + Cached Bitset Filters

```
{ "size": 3, "query": { "bool": {
    "must": [{ "match": {"description": "noise
          cancelling"} }],
    "filter": [
      { "term":  { "category": "headphones" } },
      { "range": { "price":  { "lte": 100 } } },
      { "range": { "rating": { "gte": 4.0 } } } ]
          }},
  "_source": ["name", "price", "rating"] }
```

Real Execution Output (113 hits, 2.7 ms engine):

```
"hits": [
  { "_id": "26015", "_score": 8.992849,
    "_source": { "name": "Bose QuietComfort 396",
                 "price": 80.96, "rating": 4.2 } },
  { "_id": "13815", "_score": 8.625748,
    "_source": { "name": "Audio-Technica BassBoost
         365",
                 "price": 88.24, "rating": 5.0 } },
  { "_id": "16040", "_score": 8.625748,
    "_source": { "name": "Beyerdynamic AcoustiMax
         400",
                 "price": 52.90, "rating": 4.3 } } ]
```

Demo 5: Faceted Aggregations (`doc_values`)

```
{ "size": 0, "aggs": { "by_category": {
    "terms": { "field": "category" },
    "aggs": {
      "avg_price":  { "avg": { "field": "price" } },
      "max_rating": { "max": { "field": "rating" } }
          } }} }
```

Real Execution Output (All 50,000 docs in 1.0 ms!):

Category Bucket	Doc Count	Avg Price ($)	Max Rating
cameras	8,461	2,181.53	5.0
running shoes	8,366	157.69	5.0
headphones	8,328	287.31	5.0
smartphones	8,320	1,001.72	5.0
laptops	8,280	1,985.25	5.0
coffee machines	8,245	613.51	5.0



Live Demo: Cassandra Partition Key, Clustering Range & SAI ($N = 50,000$)

Demo 1: Partition-Key Lookup ($p_{50} = 1.70$ ms)

```
CQL> SELECT name, price, rating FROM
     shop.products_by_category
     WHERE category='laptops' LIMIT 5;

 name                | price  | rating
---------------------+--------+--------
   Acer BladePro 874 | 450.26 |    3.8
   Apple ProBook 357 | 450.36 |    3.9
   Dell BladePro 453 | 450.55 |    3.6
 Lenovo BladePro 165 | 451.08 |    4.5
  Dell UltraBook 907 | 452.56 |    4.3
```

Demo 2: Partition + Clustering Range ($p_{50} = 2.31$ ms)

```
CQL> SELECT name, price FROM
     shop.products_by_category
     WHERE category='laptops' AND price>=500 AND
          price<=800 LIMIT 5;

 name                | price
---------------------+--------
 Dell UltraBook 653  | 500.63
  MSI ThinkBook 481  | 500.80
 Acer UltraBook 840  | 501.06
```

Demo 3: SAI Equality Index on `brand` ($p_{50} = 3.84$ ms)

```
CQL> SELECT name, category, price FROM
     shop.products_by_category
     WHERE brand='Sony' LIMIT 5;

 name                | category     | price
---------------------+--------------+--------
       Sony Nova 767 | smartphones  | 200.55
     Sony Xperia 787 | smartphones  | 202.27
 Sony UltraPhone 268 | smartphones  | 202.89
   Sony RedMagic 547 | smartphones  | 203.04
   Sony NoteEdge 526 | smartphones  | 203.26
```

Demo 4: Partition + SAI BKD Range ($p_{50} = 4.40$ ms)

```
CQL> SELECT name, rating, price FROM
     shop.products_by_category
     WHERE category='laptops' AND rating>=4.8 LIMIT
          5;

 name                  | rating | price
-----------------------+--------+--------
 Asus WorkStation 226  |    5.0 | 457.20
     Asus ProBook 602  |    4.8 | 457.29
    Dell BladePro 793  |    5.0 | 458.73
```

Live Demo: Cassandra Anti-Patterns & 50,000-Row Scan Penalty

1. Filtering Without Index → Refused! (Demo 5)

```
CQL> SELECT name FROM shop.products_by_category
     WHERE stock > 100 LIMIT 5;

InvalidRequest: code=2200 [Invalid query]
message="Cannot execute this query as it might involve
     data
filtering and thus may have unpredictable
     performance...
use ALLOW FILTERING"
```

2. Full-Text `LIKE` Pattern → Refused! (Demo 7)

```
CQL> SELECT name FROM shop.products_by_category
     WHERE description LIKE '%laptop%' LIMIT 3;

InvalidRequest: code=2200 [Invalid query]
message="LIKE restriction is only supported on properly
indexed columns. description LIKE '%laptop%' is not
     valid."
```

3. Forcing `ALLOW FILTERING` on 50K Rows (Demo 6)

```
CQL> SELECT name, category, stock, review_count
     FROM shop.products_by_category
     WHERE stock = 500 AND review_count > 650
     LIMIT 5 ALLOW FILTERING;

 name                          | category        |
      stk | rev
-------------------------------+-----------------+-----+-
 Nespresso EspressoMaster 284 | coffee machines |
      500 | 660
           OnePlus Galaxy 534 |      smartphones |
                500 | 680
          Asics UltraCushion 382 |   running shoes |
               500 | 765
```

Empirical Impact at $N = 50,000$

- Because only 3 rows match (< `LIMIT 5`), Cassandra must decompress and scan **100% of all 50,000 rows across both SSTables!**
- Measured latency jumps from **1.70 ms** (Partition Key) to **115.88 ms** (p_{50}) and **192.23 ms** (p_{95}).

Live Demo: Semantic Vector ANN (Cassandra 5.0) & Hybrid Search (ES 8.15)

ES Demo 10: Hybrid BM25 + HNSW Vector kNN

```
POST /products/_search
{ "size": 3, "query": { "match": { "description": {
    "query": "portable travel", "boost": 0.5 }}},
  "knn": { "field": "embedding", "k": 10,
       "num_candidates": 50,
    "query_vector":
         [0.64,0.54,0.09,0.09,0.50,0.18,0.0,0.0],
    "boost": 0.5 }, "_source": ["name", "category",
         "price"] }
```

Real Execution Output ($p_{50} = 10.55$ ms, 4.7 ms engine):

```
"hits": [
  { "_id": "13271", "_score": 1.1621861, "_source": {
      "name": "MSI AirLite 586", "price": 2869.58,
           "rating": 4.6 }},
  { "_id": "45405", "_score": 1.1621354, "_source": {
      "name": "Acer WorkStation 865", "price":
           2802.16, "rating": 3.8 }},
  { "_id": "3160",  "_score": 1.0807763, "_source": {
      "name": "Asus ThinkBook 970", "price":
           1372.59, "rating": 3.2 }}]
```

Cassandra Demo 9: SAI JVector ANN OF Search

```
SELECT name, category, price,
     similarity_cosine(embedding,
  [0.64,0.54,0.09,0.09,0.50,0.18,0.0,0.0]) AS
       sem_score
FROM shop.products_by_category WHERE
     category='laptops'
ORDER BY embedding ANN OF
     [0.64,0.54,0.09,0.09,0.50,0.18,0.0,0.0] LIMIT 5;
```

Real Execution Output ($p_{50} = 8.31$ ms):

```
 name                | category | price   |
      sem_score
---------------------+----------+---------+-----------
   Asus ThinkBook 970 |  laptops | 1372.59 |
         0.99793
       HP AirLite 496 |  laptops | 3261.94 |
            0.997846
 Asus WorkStation 110 |  laptops | 2587.89 |
      0.997543
    MSI UltraBook 338 |  laptops |  544.02 |
           0.997511
      MSI AirLite 586 |  laptops | 2869.58 |
            0.997435
```

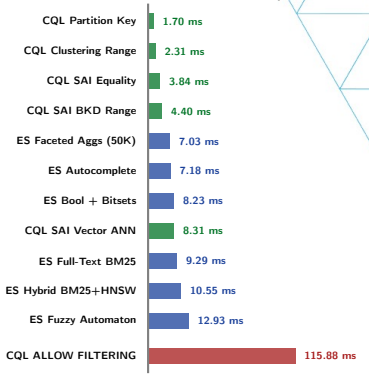
Insight: Both engines find Asus ThinkBook 970 & MSI AirLite 586 as top

Quantitative Benchmark: Latency Distribution ($N = 50,000$ Products)

Engine	Workload (25 Runs)	Mean	p50	p95	
Cass 5.0	1. Partition Key $O(1)$	1.74	**1.70 ms**	2.00	
Cass 5.0	2. Clustering Range	2.87	**2.31 ms**	6.92	
Cass 5.0	3. SAI Equality (`brand`)	4.00	**3.84 ms**	5.36	
Cass 5.0	4. Partition + SAI Range	4.43	**4.40 ms**	5.81	
Cass 5.0	5. SAI Vector `ANN OF`	9.33	**8.31 ms**	10.36	
Cass 5.0	6. `ALLOW FILTERING` (50K)	124.67	**115.88 ms**	192.23	**ES*
ES 8.15	1. Faceted Aggs (50K)	7.21	**7.03 ms**	10.01	
ES 8.15	2. Edge N-Gram Prefix	7.29	**7.18 ms**	8.39	
ES 8.15	3. `bool` + 3 Bitset Filters	9.16	**8.23 ms**	12.77	
ES 8.15	4. Full-Text BM25	9.66	**9.29 ms**	12.25	
ES 8.15	5. Hybrid (BM25 + HNSW)	12.60	**10.55 ms**	18.26	
ES 8.15	6. Fuzzy Automaton	13.82	**12.93 ms**	19.94	

*Engine took: **1.00 ms** (Aggs), **1.48 ms** (N-Gram), **2.72 ms** (Bool), **3.24 ms** (BM25), **4.72 ms** (Hybrid), **7.52 ms** (Fuzzy).*

Median Latency Comparison (p_{50} in ms)

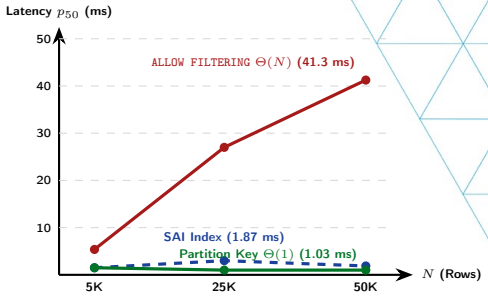


Scalability Stress Test: $N = 5,000 \rightarrow 25,000 \rightarrow 50,000$ Rows

Empirical Scaling Behavior (`benchmark.py`)

Dataset (N)	Partition Key	SAI (`brand`)	`ALLOW FILTERING`
5,000	1.56 ms	1.55 ms	5.38 ms
25,000	0.95 ms	2.94 ms	27.00 ms
50,000	1.03 ms	1.87 ms	41.29 ms
50,000 (SSTable)	1.70 ms	3.84 ms	**115.88 ms**

Query Latency vs. Dataset Size (N)



Complexity Verification

- **Partition Key Lookup ($\Theta(1)$):** Murmur3(`category`) jumps to 1 partition and stops after `LIMIT 5` rows. Latency is invariant to N.
- **SAI Trie Index ($O(\log V + K)$):** Traverses Byte-Ordered Trie for `'Sony'` and fetches top-5 postings.
- **`ALLOW FILTERING` ($\Theta(N)$):** Scales linearly with N (5.38 ms $\rightarrow 41.29$ ms in RAM; 115.88 ms decompressing SSTables + vectors on disk). At $N = 50$M, latency exceeds **110 s** (timeout)!

Table of Contents

1. Introduction & Search Fundamentals
2. Elasticsearch 8.15: Distributed Full-Text & Hybrid Search
3. Apache Cassandra 5.0: Scalable Wide-Column Store & SAI
4. Modern Research Frontier & Combined Architecture
5. Experimental Demo & Quantitative Benchmark ($N = 50,000$)
6. **Conclusion**



Conclusion & Key Takeaways

1. Elasticsearch 8.15 = Discovery & Relevance

- Answers **unstructured & semantic user queries** in 1–8 ms via Lucene inverted indexes, FSTs, custom analyzers, and HNSW vector graphs.
- Delivers **Okapi BM25 ranking**, Levenshtein typo tolerance, highlighting, and 1.0 ms faceted aggregations over $50,000$ docs via columnar `doc_values`.

2. Cassandra 5.0 = Predictable Linear Scale

- Answers **known, high-velocity access patterns** in 1.70 ms ($O(1)$ routing) via consistent hashing and append-only LSM-Tree partitions.
- Cassandra 5.0 **SAI** adds low-overhead Trie, BKD-tree, and JVector ANN filtering, while guarding cluster SLAs against full-scan anti-patterns.

3. Graduate Engineering Synthesis

- **Data Structures Dictate Capabilities:** Inverted Index + HNSW vs. Partitioned LSM-Tree + SAI represent fundamentally complementary trade-offs.
- **Production Standard:** Pair **Cassandra as the authoritative source of truth** (`LOCAL_QUORUM` + LWT checkout) with **Elasticsearch as the discovery engine**, synced via Kafka CDC.

Complete Reproducible Artifact: Docker stack, 50,000-product generator with 8D embeddings, 20 guided live queries, and automated benchmark suite included in repository.



References I

- Apache Software Foundation. *Apache Cassandra 5.0 Documentation*. 2024. URL: https://cassandra.apache.org/doc/latest/.
- — .*Storage-Attached Indexing (SAI) in Cassandra 5.0*. 2024. URL: https://cassandra.apache.org/doc/latest/cassandra/indexing/sai/sai-overview.html.
- Cormack, Gordon V., Charles L. A. Clarke, and Stefan Buettcher. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods". In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, pp. 758–759.
- Elastic N.V. *Elasticsearch Reference 8.15*. 2024. URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html.
- Gormley, Clinton and Zachary Tong. *Elasticsearch: The Definitive Guide*. O'Reilly Media, 2015.
- Karger, David et al. "Consistent Hashing and Random Trees". In: (1997).

References II

- Lakshman, Avinash and Prashant Malik. "Cassandra: A Decentralized Structured Storage System". In: *ACM SIGOPS Operating Systems Review*. Vol. 44. 2. 2010, pp. 35–40.
- Malkov, Yu A. and Dmitry A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836.
- O'Neil, Patrick et al. "The Log-Structured Merge-Tree (LSM-Tree)". In: vol. 33. 4. 1996, pp. 351–385.
- Robertson, Stephen and Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.
- Subramanya, Suhas Jayaram et al. "DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. 2019.
- Zobel, Justin and Alistair Moffat. "Inverted Files for Text Search Engines". In: vol. 38. 2. 2006.

- THANK YOU -

Questions & Answers (Q&A)

Course: Data Engineering (CO5173) – Semester 261
Supervisor: Assoc. Prof. Trần Minh Quang
Students: Đặng Kiến Tường (2670391) – Đặng Minh Quân (2670358)
Live Stack: Elasticsearch 8.15 (:9200) · Kibana (:5601) · Cassandra 5.0 (:9042)